

Understanding Parallel Performance

How do we know we've succeeded?

By Herb Sutter

Herb is a bestselling author and consultant on software development topics, and a software architect at Microsoft. He can be contacted at www.gotw.ca.

Let's say that we've slickly written our code to apply divide-and-conquer algorithms and concurrent data structures and parallel traversals and all our other cool tricks that make our code wonderfully scalable in theory. Question: How do we know how well we've actually succeeded? Do we really know, or did we just try a couple of tests on a quad-core that looked reasonable and call it good? What key factors must we measure to understand our code's performance, and answer not only whether our code scales, but quantify how well under different circumstances and workloads? What costs of concurrency do we have to take into account?

This month, I'll summarize some key issues we need to keep in mind to accurately analyze the real performance of our parallel code. I'll list some basic considerations, and then some common costs. Next month, I have a treat in store: We'll take some real code and apply these techniques to analyze its performance in detail as we successively apply a number of optimizations and measure how much each one actually buys us, under what conditions and in what directions, and why.

Fundamentals

To understand our code's scalability, we need to know what to measure and what to look for in the results. First, identify the workload variables: What are the key different kinds of work and/or data? For example, we may want to measure how well a producer-consumer system performs with varying numbers of producer threads and consumer threads, or measure a container by how well it scales when holding data items of different sizes.

Second, use stress tests to measure throughput, or the total amount of work accomplished per unit time, while varying each of these dimensions so that we can measure their relative impact. Look for scalability trends, the change in throughput as we add hardware resources: Can we effectively use more cores to either get the answer faster or to get more work done?

Figures 1 and 2 show two useful visualization tools that will help us understand our code's parallel performance. Figure 1 shows a sample scatter graph that charts throughput results for a selected algorithm against different numbers of two kinds of workers: producer threads and consumer threads. The larger the bubble, the greater the throughput. We can get a sense of how this particular algorithm scales, and in what directions, by examining how and where throughput grows and shrinks. In this example, we have good scalability up to a total of about 15 threads before we start to peak and realize no further gains, and we can see that scalability is better when there are somewhat more producers than consumers in the system.

[Click image to view at full size]

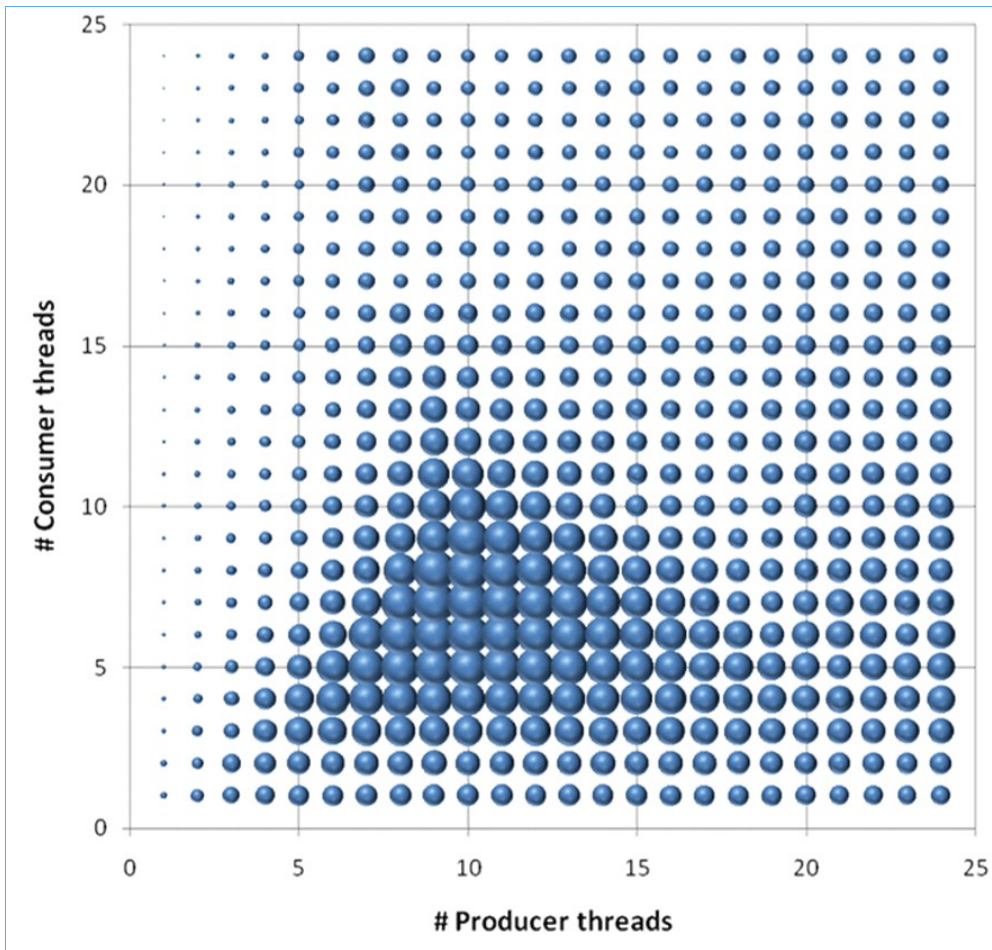


Figure 1: Sample graph for measuring scalability of the same algorithm for different workloads.

Figure 2 directly compares different candidate algorithms running the same kind of workload. Peak throughput occurs, naturally enough, at the peak of each curve. Scalability shows up directly as the left-hand ascending side of the curve; the steeper it is, and the farther to the right that it goes before topping out, the more scalable our code will be. Here, the blue algorithm demonstrates the horror of negative scalability; it actually manages to get less work done using additional cores, which probably won't earn us our next raise.

[\[Click image to view at full size\]](#)

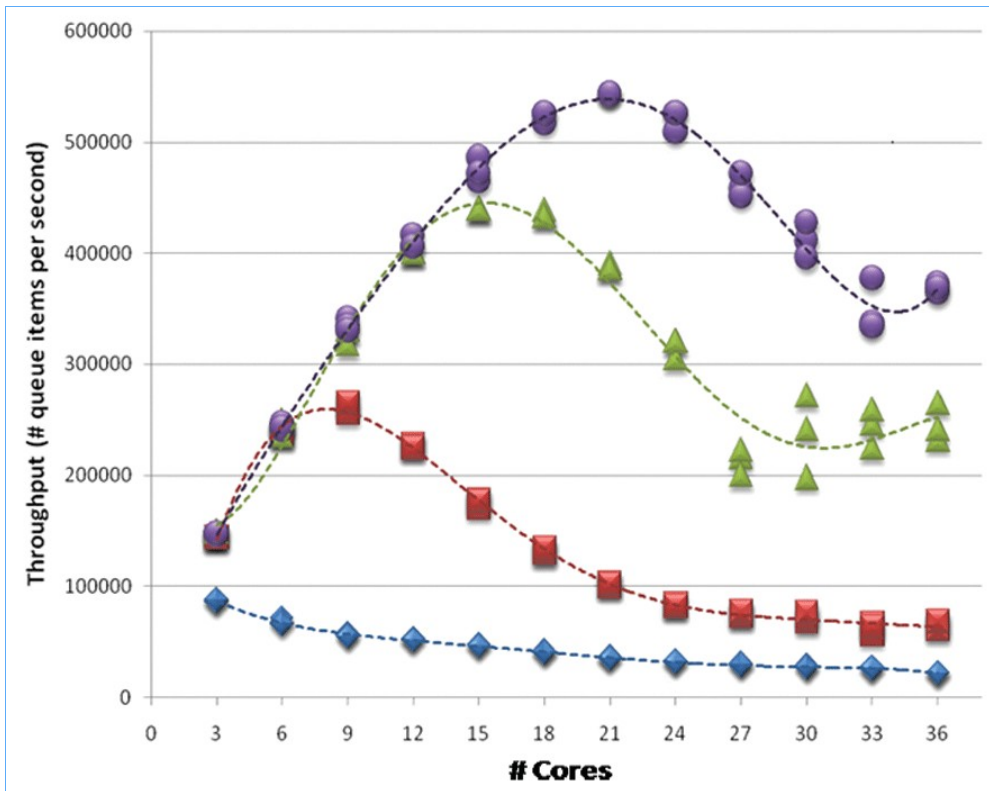


Figure 2: Sample graph for measuring scalability of alternative algorithms for the same workload.

But both figures also let us see two basic penalties. Contention arises when different workers interfere with each other by fighting for resources, such as contending for mutexes, cache space, or cache lines via false sharing. In the most extreme case, adding a new worker might actually cost more in contention than it adds in extra work, resulting in less total work being done, and we can see this extreme effect in both graphs: In Figure 1, in several directions we reach areas where adding more workers makes total throughput actually go down. In Figure 2, we see the same effect in the form of the right-hand downslope where adding more work actually decreases throughput even when there are otherwise-idle cores. But Figure 2 also lets us more clearly see the effect of contention before it gets that far: On the left-hand upslope, even as throughput is still rising, the rate of increase is slowing down as the curve begins to bend. That's a classic effect of growing contention.

The other basic penalty is oversubscription, or having more CPU-bound work ready to execute than we have available hardware. These samples were taken from test runs on a 24-core machine; sure enough, in Figure 1 we see a faint diagonal line where $\#producers + \#consumers = 24$, above which throughput is noticeably thinner; sometimes the result is even more dramatic. Similarly, in Figure 2 we see even the best algorithms can't scale beyond the available cores, and incur a penalty for trying to exceed that number by adding contention at least for CPU time and often also for other resources.

With these fundamentals in mind, let's consider a few specific costs that arise and impact scalability because of contention, oversubscription, and other effects.

Sources of Overhead, and Threads versus Pools

We incur a basic concurrency overhead from just expressing code in a parallel way. Consider the following toy example that performs three independent

subcomputations to generate a result:

```
// Example 1: Sequential code in MyApp 1.0
//
int NetSales() {
    int wholesale = CalcWholesale();
    int retail = CalcRetail();
    int returns = TotalReturns();
    return wholesale + retail - returns;
}
```

Assume that this code is entirely CPU-bound, and that the subcomputations really are independent and free of side effects on each other. Then we can get the answer faster on more cores by running the subparts in parallel. Here's some pseudocode showing how to accomplish this using futures and the convenience of lambda functions, but with a common mistake:

```
// Example 2 (flawed): Naïve parallel code for MyApp 2.0?
//
int NetSales() {
    // perform the subcomputations concurrently
    future<int> wholesale = new thread( [] { CalcWholesale(); } );
    future<int> retail = new thread( [] { CalcRetail(); } );
    future<int> returns = new thread( [] { TotalReturns(); } );

    // now block for the results
    return wholesale.value() + retail.value() - returns.value();
}
```

Seeing the words *new Thread* explicitly in code is often an indicator that code may not be as scalable as it could be. In most environments, it's much less efficient to spin up a new thread for each piece of work than to run it on a thread pool: First, spinning up a new thread and throwing it away again each time incurs substantially more overhead than giving work to an existing pool thread. Second, spinning up a number of threads and turning them loose to fight for available cores via the operating system scheduler can cause needless contention when we spin up more work than there are cores currently available, which can happen not only on low-core hardware but also on many-core hardware if the application or the system happens to be doing a lot of other work at that moment. Both situations are different kinds of oversubscription, and some threads will have to incur extra context-switching to interleave on an available core. Instead, sharing the core by running one after the other would be both more efficient and more cache-friendly.

Thread pools address these problems because a pool is designed to "rightsize" itself to the amount of hardware concurrency available on the machine. The pool will automatically try to maintain exactly one ready thread per core, and if there is more work than cores the pool naturally queues the work up and lets the machine concentrate on only as many tasks at a time as it has cores to perform. Here's pseudocode for a revised version that uses pools instead:

```
// Example 3 (better): Partly fixed parallel code for MyApp 2.0
//
int NetSales() {
    // perform the subcomputations concurrently
    future<int> wholesale = pool.run( [] { CalcWholesale(); } );
    future<int> retail = pool.run( [] { CalcRetail(); } );
    future<int> returns = pool.run( [] { TotalReturns(); } );
}
```

```
// now block for the results
return wholesale.value() + retail.value() - returns.value();
}
```

The good news is that this can enable us to get the answer faster on more cores, at least up to three cores. But there are costs too: With today's thread pools, we typically pay a tax of two context switches when we ship work over to a pool thread and then ship the result back. How can we reduce this cost?

Reducing Context Switches

We can eliminate some of the context switches by being smarter about the last line that combines the results, because just calling `.value()` three times may wake up the calling thread twice only to immediately have to sleep again; instead, use a "wait for a group of futures" facility if your futures library has one, as a well-written version can eliminate the needless wakeups.

We can also avoid a switch by observing that the calling thread isn't going to do anything anyway besides wait for the results, and so it's often a good idea to keep the tail chunk of work and do it ourselves instead of needlessly idling the original thread.

Example 4 shows both techniques in action:

```
// Example 4: Improved parallel code for MyApp 2.0
//
int NetSales() {
    // perform the subcomputations concurrently
    future<int> wholesale = pool.run( [] { CalcWholesale(); } );
    future<int> retail = pool.run( [] { CalcRetail(); } );
    int returns = TotalReturns(); // keep the tail work

    // now block for the results--wait once, not twice
    wait_all( wholesale, retail );
    return wholesale.value() + retail.value() - returns;
}
```

Naturally, what matters most is not the total overhead, but how big it is compared to the total work. We want the cost of each chunk of work to be significantly larger than the cost of performing it asynchronously instead of synchronously.

The Cost of Unrealized Concurrency

A key cost in today's environments is the cost of unrealized concurrency. What's the cost of our parallel code compared to the sequential code in the case when the parallel algorithm actually ends up running sequentially? For example, what happens if our parallel-ready code executes on a machine with just one core, so that we don't actually get to realize the concurrency because the tasks end up running sequentially anyway (e.g., there's only one pool thread)? We've added the overhead to express concurrency, and we pay for that overhead even if we don't get to benefit from it on a particular system.

If Example 1 is the code we shipped in MyApp version 1.0 and Example 4 is what we'll ship in MyApp 2.0, then an existing customer with a legacy single-core machine may find that the new application is actually slower than the old one, even though the new application will run better when more cores are available. To mitigate this on low- and single-core machines, we can reduce the overhead by adjusting granularity to use fewer and larger chunks of work, or even switch to a sequential implementation.

The Double-Edged Sword of Fine-Grainedness

Even in Example 4, the code only scales to at most three cores. Can we do better? Yes, if we can make the work more fine-grained, slicing the work to be done more finely into smaller chunks. One approach in Example 4 would be to apply similar techniques within *CalcWholesale*, *CalcRetail*, and *TotalReturns* to further decompose their work into concurrent subtasks. Even better is when we can exploit natural parallelism in algorithms (e.g., divide-and-conquer algorithms like quicksort) and data structures (e.g., trees and graphs) to subdivide our work in a way that scales with the amount of data.

But now we encounter a fundamental tension between scalability and overhead: The smaller the chunks of work, the more chunks we have and the more easily we can distribute them to utilize larger number of cores to get an answer faster—but the more the overhead per chunk starts to dominate in our performance costs.

Let's consider quicksort as a common example. Can you spot the performance flaws in this code?

```
// Example 5 (flawed, today): Naïve parallel quicksort,
// sorts left and right subranges in parallel
//
void ParallelQuicksort( Iterator first, Iterator last ) {
    Iterator pivot = Partition( first, last );
    future f1 = pool.run( [&]{ ParallelQuicksort( first, pivot ); } );
    future f2 = pool.run( [&]{ ParallelQuicksort( pivot, last ); } );
    wait_all( f1, f2 );
}
```

We can improve this in at least two ways. First, as noted earlier, we should take the tail chunk of work and do it ourselves to mitigate the overhead of shipping work to a pool thread in today's environments. Second, we can notice that most of the spun-off work will be at the leaves of the computation containing subranges of a few elements or even just one. Even in sequential code, it's typical to switch to something like bubble sort when the subrange's size falls below a threshold; similarly in parallel code, for small ranges we should switch to a sequential sort. Example 6 incorporates both of these changes:

```
// Example 6: An improved parallel quicksort, still
// sorts subranges in parallel but more efficiently
//
void ParallelQuicksort( Iterator first, Iterator last ) {
    if( distance(first,last) <= threshold ) {
        SequentialSort( first, last );
    } else {
        Iterator pivot = Partition( first, last );
        future f1 = pool.run( [&]{ ParallelQuicksort( first, pivot ); } );
        ParallelQuicksort( pivot, last );
        f1.wait();
    }
}
```

In general, we want to slice the work as finely as possible, but not to the point where the work is comparable in size to the overhead.

Forward-Looking Note: Work Stealing

Future runtime systems will significantly drive down all of these costs, including the overhead per chunk and the cost of unrealized concurrency, to the point where we will often be able to ignore it and blithely write code like Example 5 without worrying about its performance most of the time.

The basic idea is to use work stealing whereby default "potentially asynchronous work" is actually not shipped elsewhere, but rather queued up to be executed on the original thread. Only if another core runs out of work and sees that our thread has waiting queued work to be performed, will the work be "stolen" and efficiently shipped to the other thread. The idea is to drive down the cost of unrealized concurrency by only actually incurring the overhead of running on another core if it's worth doing at that particular instant—on this hardware and with this amount of other work currently occupying the machine; and the very next execution of the very same function on the very same hardware might not steal, if all the cores are already busy. Sample current and upcoming technologies that feature work stealing runtimes include: Intel Threading Building Blocks; Microsoft's Parallel Patterns Library, Task Parallel Library, and PLINQ; Java 7's Fork/Join framework; and the granddaddy of them all, Cilk, which popularized the technique among implementers.

On Deck

To understand your code's scalability, first identify the key variables that affect the workload, and then measure throughput for workloads with different combinations of those variables. Look for scalability, and how it hits the contention and oversubscription barriers. Prefer thread pools (today) and work stealing (tomorrow).

Next month, we're going to apply the tools we discussed this month to analyze the performance impact of specific optimizations on concrete code. Fasten your seat belts.